Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

Jul 28, 2020 · 8 min read · ✦ · Listen

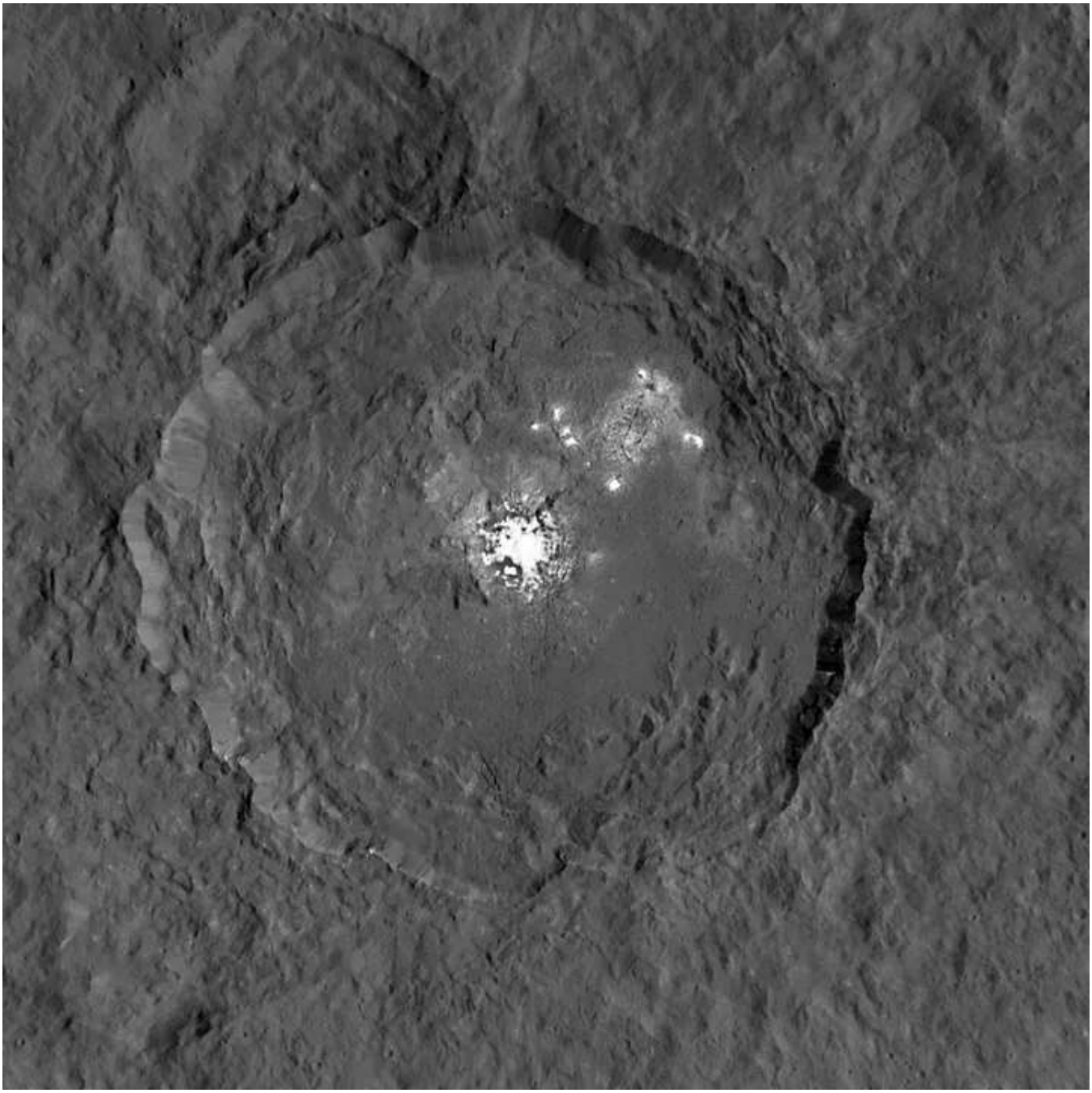
Save

SPACE SCIENCE WITH PYTHON

Space Science with Python — Ceres in the Sky

Part 20 of the tutorial series links miscellaneous concept from the last tutorials to understand the movement and brightness development of the largest asteroids in the main belt: (1) Ceres

63



The brightest spot of Ceres, embedded in the Occator Crater. Image taken by the Dawn spacecraft. Credit: NASA/JPL-Caltech/UCLA/MPS/DLR/IDA; License: Creative Commons Attribution 4.0 International License; From: ESO

Preface

This is the 20th part of my Python tutorial series "Space Science with Python". All codes that are shown here are uploaded on GitHub. Enjoy!

What we learned so far

I hope you enjoyed the last couple of weeks, where you learned the basic principles, methods, Python libraries and theoretical concepts that are used in *Space Science*.

Our overall objective: To become a Citizen Space Scientist, focusing on interesting Solar System topics.

In the beginning, we learned how to setup and install our required Python modules; especially the NASA toolkit SPICE. SPICE is being used by engineers and scientists to plan and predict e.g., spacecraft trajectories and instrument orientations. It can also be used to determine the positions and velocity components of objects like planets, asteroids and so on.

Space Science with Python — Setup and first steps

A starting guide to became a Citizen Space Scientist

towardsdatascience.com

Scientific topics were covered by the article series, too, like the movement and phase angle computations of Venus a few weeks ago …

Space Science with Python — The dance of Venus

Part 4 of the tutorial series takes a look how to determine a perfect timing for a great Venus & Moon photo

towardsdatascience.com

… the determination and visualisation of sky coordinates …

Space Science with Python — Space maps

Part 5 of the tutorial series shows how to calculate and understand coordinate systems that are used for all upcoming…

towardsdatascience.com

… or the computation of the apparent magnitude (brightness) of asteroids, based on commonly used equations.

Space Science with Python — Bright Dots in the Dark Sky

Part 16 of the tutorial series describes another important basic concept in Space Science: the brightness of objects.

towardsdatascience.com

These equations are empirically derived. A simple "look" at the formula does not allow one to derive anything. Equations need to be studied to fully understand the effects they describe like e.g., the Opposition Effect.

Space Science with Python — A very bright Opposition

Part 19 of the tutorial series dives deeply into asteroid research: How does the brightness of an asteroid change? What...

towardsdatascience.com

We have covered topics that can be linked together, allowing us to dive deeper into scientific topics. *Today, we link the apparent magnitude , phase angle and distance computations together. And we will plot the results of Ceres' movement and brightness changes as seen from Earth for the entire year 2020.*

I think, with today's lesson we are ready to aim for our very first science project that will be briefly described at the end.

Linking knowledge

Let's begin. First, we start with importing all necessary modules that we used in the last sessions. Since we need to determine distances and phase angles between objects, we import some SPICE kernel files via a kernel meta file (line 8). The equations for the apparent magnitude (see the session from last time) are stored in an auxiliary Python script that is loaded in line 13 and 14. ***Please note: Download the spk kernel codes_300ast_20100725.bsp manually from the NAIF page and store it in the project's folder _kernels/spk.***

According to the asteroid spk kernel *codes_300ast_20100725.bsp* documentation the Ceres NAIF ID is 200001 (line 2). For the apparent magnitude computations we need also two intrinsic properties of Ceres:

- The Absolute Magnitude (line 7)
- The Slope Parameter G (line 8)

Both parameters can be found in the NASA / JPL Small Body Database:

JPL Small-Body Database Browser

hide orbit diagram] [revert to old Java-based viewer] Orbital Elements at Epoch 2458600.5 (2019-Apr-27.0) TDB...

ssd.jpl.nasa.gov

Part 2/9

All results shall be stored in a dataframe (line 2) and the time range of our computations shall be the year 2020. Let's set a date-time for each week of the year (line 7). For the plotting routine later, we will plot the Day-Of-Year (*%j* in the *strftime* function) that is being generated and stored in line 13.

Finally, we convert the date-times to their corresponding Ephemeris Times (ETs) in line 16 and 17 using the SPICE function utc2et.

Part 3/9

For the apparent magnitude computation we need the distance between Ceres and the Sun / and the Earth, as well as the phase angle (coding part 5/9). For the distance computation we use the SPICE function spkgps; the resulting position vector is used within vnorm to determine the norm (length) of the vector; finally convrt is used to convert the distance, given in km, to AU.

We apply the functions twice for Ceres — SUN (line 3 to 11) and Ceres — Earth (line 15 to 23).

Part 4/9

The phase angle can easily be computed by using the function phaseq. The function requires the following inputs:

- **et**: The ET of the required computation time
- **target**: The target object (here: the Ceres ID)
- **illmn**: The illumination object (here: the Sun)
- **obsrvr**: The observer object (here: the Earth)
- **abcorr**: Any aberration correction. Here, we apply no correction

Please note 2 things: first, the NAIF IDs are not integers, but strings this time. The function cannot handle any numerical values. Second, the input parameter *target*, *illmn* and *obsrvr* appear to be confusing: *how are the terms "target" and "observer" defined?*

A simple sketch in the SPICE documentation provides an answer of the definitions:

```
                 ILLMN      OBS
ILLMN as seen      ^        /
from TARG at       |       /
ET - LT.           |      /
                  >|..../< phase angle
                   |    /
                 . |   /
               .   | /
              .    |v     TARG as seen from OBS
        SEP   .   TARG    at ET
               .  /
                 /
                v

 PI = SEP + PHASE

 so

 PHASE = PI - SEP
```

Line 10 and 11 convert the angles from radians to degrees.

Part 5/9

We have now all parameters that are needed to compute the apparent magnitude of Ceres:

- Distance between Ceres and the Earth
- Distance between Ceres and the Sun
- Phase angle between Sun and Earth as seen from Ceres
- Slope parameter G of Ceres
- Absolute magnitude of Ceres

For every time step line 2 to 9 compute the apparent magnitude. The function *app_mag* was introduced last time:

Part 6/9

The corresponding sky coordinates shall be computed in the ecliptic J2000 reference frame. Thus, we determine the positional vector of Ceres in *ECLIPJ2000* and compute the longitude and latitude values in line 2 to 7 and 10 to 15, respectively, using recrad. Finally, the radians values are converted to degrees (line 18 and 19 as well as line 21 and 22).

Part 7/9

How shall we visualise the data? Well, we have the sky coordinates and the brightness values that can be put into one single sky map (or section of a sky map). The coordinates corresponding brightness values could scale the dot size to get a better

feeling how the apparent magnitude changes over time. However, a small apparent magnitude corresponds to a brighter object. Scaling the dots with the magnitude values would be somehow contra-intuitive: Brighter appearances would lead to smaller dots. Let's change that by introduced a simple scaling method that inverts this problem.

We will use scikit-learn in a moment, so we import it first (line 9). Line 13 and 14 scale the magnitude values based on the minimum magnitude value. For example: a minimum magnitude of 5 would become 1, larger magnitudes become smaller than 1.

The resulting values are now scaled between 0 and 1. For this purpose, we use the *MinMaxScaler* (line 17), fit the scaler (line 20) and transform our "pre scaled" values in line 23. Now we multiply a constant to the resulting array that ranges between 0 and 1 to improve the dot size visibility. The result is stored in the dataframe (line 30).

Part 8/9

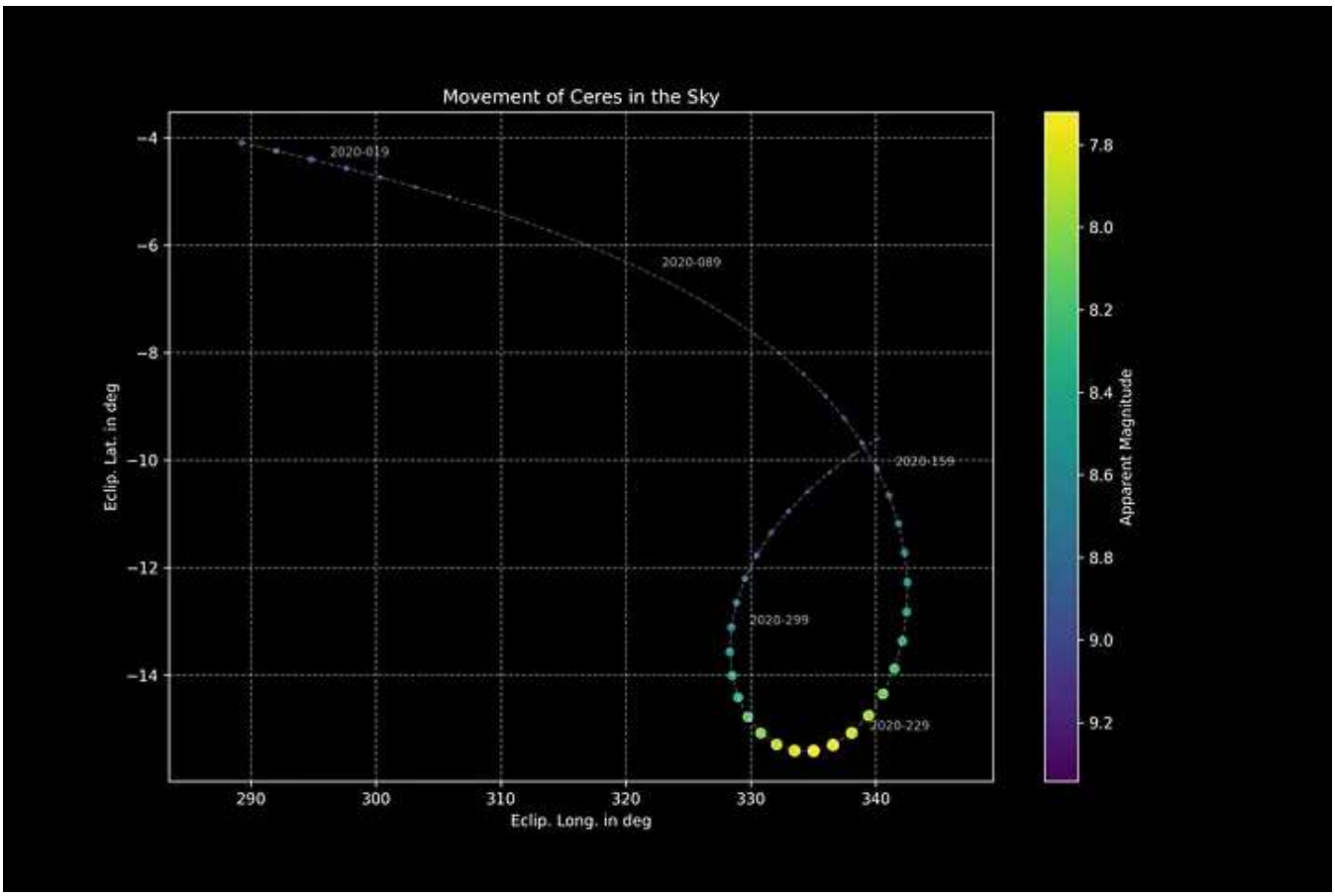
We are ready to plot the results! A scatter plot is used (line 12 to 18) to visualise Ceres' position at each time step. The dots are linked with a white dashed line (line 21 to 27) to improve the readability. Now, with a for-loop between line 31 and 44 we add the date-time string for every 10th time step. This helps us to see the movement direction of Ceres.

Since a smaller magnitude corresponds to a brighter appearance, we invert the colorbar of the scatter plot in line 56. After some more formatting, we store the resulting plot. Let's take a look:

Part 9/9

You can see a section of a sky map, given in Ecliptic J2000 coordinates. The ecliptic latitude is plotted vs. the ecliptic longitude in degrees. The path shows the trajectory of Ceres as seen from our home planet and the dots represent different date-times (once per week). The dot size scales clearly with the apparent magnitude that is also colour coded. The apparent magnitude ranges from around 7.6 to 9.4 mag; covering almost 2 orders of magnitude.

The movement of Ceres appears at some point "retrograde". At the same time, the brightness increases, reaches its maximum and decreases afterwards. This apparent motion is particular for objects during their opposition phase with respect to Earth. This "Opposition Loop" is only an observational effect and does not correspond to any movement changes. In a future, supplementary article, we will animate this motion in more depth.



Movement of Ceres in Ecliptic Coordinates. The trajectory applies for observations as seen from Earth for the entire year 2020. Each dot represents the position of Ceres for a particular year's week. The size and colour correspond to the apparent brightness of the object. Credit: T. Albin

In the meantime, the YouTube channel *Vox* has a nice explanatory video of this topic:

Next Step: An Asteroid Science Project

I hope you have some time to digest all topics, methods, programming parts and the theory from the last 20 tutorials! And I also hope that you enjoy reading it as much as I enjoy writing these tutorials. When I worked in academia, public outreach, education, given lectures etc. was a ton of fun! And I want to continue with it.

With the knowledge you have now we can aim for our very first space-scientific project. It will be about Asteroids and Near-Earth Objects. We will do some research, programming, analysis and interpretation in a series of articles that will be linked. Together, we will work on a scientific project to estimate the detectability and visibility of *all* potentially hazardous objects. We will use actual data and model data from the literature to answer the question, how many objects are out there …. *hidden* … but dangerous? And is there an effective way to improve the detection of yet unknown objects? Next time, we will start with this topic from a theoretical perspective.

In the meantime: Stay Curious

Thomas

Space Science Python Programming Space Science With Python

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play